DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40I

USAREUR-AF

CONCEPTS GUIDE — TM-40I COMPANION

MACHINE LEARNING ENGINEER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

CONCEPTS GUIDE — TM-40I COMPANION

MACHINE LEARNING ENGINEER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany | 2026

PURPOSE: Develops the mental models required to build, deploy, and maintain ML models on MSS effectively. Read before beginning TM-40I task instruction.

DISTRIBUTION RESTRICTION: DRAFT — Not yet approved for distribution.

TABLE OF CONTENTS

1. The MLE's Role on MSS
2. Decomposing an Operational ML Problem
3. Feature Engineering as Domain Modeling
4. Training Data in an Operational Context
5. Model Evaluation — Operational Metrics vs. Statistical Metrics
6. Deployment as a System, Not an Event
7. Model-Backed Ontology Properties — The Foundry-Specific Mental Model
8. Bias and Fairness in Operational Models
9. Common MLE Failure Modes on MSS

DRAFT

SECTION 1 — THE MLE'S ROLE ON MSS

BLUF: The ML Engineer builds, validates, and maintains production ML models that generate operationally useful predictions. The MLE owns the full model lifecycle — from raw feature engineering through deployed drift detection. That ownership does not end at deployment.

1-1. Role Boundaries

Comparison	MLE vs. Adjacent Role
ORSA vs. MLE	The ORSA applies quantitative methods to support a specific command decision — often a one-time analysis artifact. The MLE builds a continuously-operating system generating predictions at scale. An ORSA builds a readiness model to answer a specific question once. An MLE builds a readiness prediction pipeline that scores every vehicle in the theater every day. Both are valuable. Neither replaces the other.
AI Engineer vs. MLE	The AIE integrates LLMs, agent frameworks, and RAG into operational workflows. The MLE's tools are statistical learning, feature pipelines, and model validation. The MLE owns the predictive ML component; the AIE owns the orchestration layer around it.
Software Engineer vs. MLE	The SWE writes application code: front-end logic, API services, data infrastructure. The MLE is accountable for the model artifact and its behavior in production — not the surrounding application code.

1-2. The MLE Lifecycle

Phase	MLE Responsibility
Problem scoping	Determine whether ML is the right tool
Data access	Identify authorized training data sources
Feature engineering	Translate domain knowledge into numerical features
Model training	Select algorithm, train, cross-validate
Evaluation	Validate against operational performance criteria
Governance review	Submit documentation for C2DAO coordination
Deployment	Deploy to MSS/Foundry with monitoring hooks
Monitoring	Operate drift detection; respond to alerts
Retraining	Trigger, re-validate, re-coordinate before updating production
Decommission	Remove model and archive documentation

DRAFT

No phase is optional. An MLE who builds an excellent model and deploys it without drift detection has completed roughly half the job.

1-3. The Cultural Reality

On MSS, ML models generate properties on Ontology objects that real Soldiers and real commanders see in real time. A readiness prediction score attached to a vehicle in the Foundry Ontology is not an academic exercise. An incorrect prediction may cause a unit to deploy a vehicle about to fail, or to hold a vehicle that is fully mission-capable. The MLE is not insulated from those consequences by the model's statistical framing. Own the output.

SECTION 2 — DECOMPOSING AN OPERATIONAL ML PROBLEM

BLUF: Before writing a single line of code, determine whether ML is actually the right tool. Most operational problems do not require ML. Applying ML because it is available — not because it is necessary — wastes effort and degrades trust in the platform.

2-1. The Decomposition Framework

Apply this five-step framework to every incoming ML request:

Step 1: What is the commander's decision requirement? Start here, not with the data. What specific decision must be made, and how often? "The G4 needs to anticipate which wheeled vehicles will require unscheduled maintenance within 30 days so the theater sustainment brigade can pre-position repair parts" — that is a precise decision requirement. "Improve readiness visibility" cannot be operationalized.

Step 2: What type of decision is this?

Decision Type	Description	Example
Classification	Is this instance in class A or B?	Will this vehicle need unscheduled maintenance in 30 days?
Ranking	Which instances should be prioritized?	Which 20 vehicles are at highest risk this week?
Regression	What quantity is expected?	How many labor-hours will this maintenance event require?
Anomaly detection	Is this instance outside normal bounds?	Is this fuel consumption reading anomalous?
Clustering	Which instances are similar?	Which units have similar readiness patterns?

Step 3: Does ML add value over simpler methods?

ML Adds Value When	ML Does NOT Add Value When
The relationship between inputs and output is nonlinear and complex	A simple threshold or rule captures the pattern adequately
The number of relevant variables makes manual rule-setting impractical	Historical data does not reflect the future operational environment
Historical patterns genuinely predict future outcomes	The problem is better addressed by data quality improvement than by modeling
The volume of predictions required exceeds what manual analysis can produce	There is insufficient labeled historical data to train and validate a model

Step 4: What does correct look like? Define success in operational terms before building. "Correctly identifies 80% of vehicles requiring unscheduled maintenance, with fewer than 15% false positives" is a success criterion. "High accuracy" is not.

Step 5: What does failure look like? Define the failure case explicitly. A missed vehicle needing maintenance (false negative) can cause a breakdown during high-tempo operations. An incorrectly flagged vehicle (false positive) wastes maintenance resources. In sustainment contexts, false negatives carry higher operational risk — design the model's threshold to reflect that asymmetry.

2-2. Decision Table: ML vs. Alternatives

Condition	Recommended Approach
Small data (<500 labeled examples), clear logic	Rule-based system or simple threshold
Statistical relationship well-understood, linear	ORSA regression model
Need to predict a continuous value from structured data	Classical regression (MLR, gradient boosting)
Classify from structured tabular data	Gradient boosting classifier (XGBoost, LightGBM)
Complex nonlinear patterns, large labeled dataset	Neural network (with governance review)
Pattern recognition in unstructured data (text, imagery)	Deep learning (requires AIE coordination)
Time-series with seasonal patterns	ARIMA, Prophet, or LSTM (evaluate per use case)

VIGNETTE: The G3 requests a predictive model to forecast which units will miss deployment readiness thresholds. On examination, 95% of units that missed thresholds had already filed exception-to-policy (ETP) requests 30 days prior. A rule-based query — "flag units with open ETPs and readiness below

85%" — captures nearly the entire at-risk population. Building a gradient boosting model adds no predictive value and creates governance burden. The correct answer is a filtered Ontology view, not an ML model.

SECTION 3 — FEATURE ENGINEERING AS DOMAIN MODELING

BLUF: Features are not columns. Features are operationalized hypotheses about what drives the outcome. Most model quality is won or lost in feature engineering, not in algorithm selection.

3-1. What a Feature Actually Is

A feature is a claim: "I believe this measurement is causally or predictively related to the outcome I am trying to model." Every feature encodes an assumption about the operational world. When a model fails, the failure is almost always traceable to a wrong feature — a wrong assumption about what drives the outcome.

3-2. The Domain Knowledge Translation Problem

An experienced S4 NCO with 10 years on Stryker fleets knows: - Vehicles above 80% utilization for three consecutive months likely have deferred maintenance not in the records - Vehicles in the Northern corridor see heavier suspension wear due to road conditions - Units that recently rotated personnel have higher rates of operator error - Cold-weather starts below -15°C correlate with battery and hydraulic system issues in the following weeks

None of this knowledge exists as a column in the database. The MLE's job is to engineer features that capture these dynamics: - Rolling 90-day utilization rate (aggregation over time) - Operating location flag (geographic feature derived from unit assignment) - Personnel rotation rate in the past 60 days (joined from personnel data) - Count of days below -15°C in the past 30 days (joined from meteorological data)

The MLE who builds a model from raw database columns without consulting subject-matter experts will produce an inferior model regardless of the algorithm used.

DRAFT

3-3. Feature Categories on MSS

Feature Category	Description	Construction Approach
Raw measurements	Direct sensor or database values	Minimal transformation; validate range and freshness
Temporal aggregations	Rolling averages, cumulative counts, time-since-event	Window functions over time-indexed data
Ratio features	Normalize by base quantity to remove scale confounding	Division with zero-division handling
Cross-entity joins	Attributes from related Ontology objects	Foundry Ontology link traversal
Lag features	Prior value at T-N for time series	Shift operations on time-indexed datasets
Derived domain features	Engineered from domain knowledge	Domain consultation + deliberate construction
External context	Weather, calendar, exercise schedule	Authorized external data sources only

3-4. The Feature Validation Checklist

Before a feature enters a production model: - ☐ Does this feature have a plausible causal or correlational relationship to the target? Can you explain it to the S4? - ☐ Is this feature available at prediction time? (No future-data leakage) - ☐ Is this feature computed consistently between training and inference pipelines? - ☐ Does this feature contain proxy information for a protected characteristic? (See Section 8) - ☐ What is the missing-data rate, and how is missingness handled? - ☐ Is the feature computed from authorized data sources?

3-5. Algorithm Selection

Once feature engineering is complete, algorithm selection is secondary. Given a well-engineered feature set and sufficient labeled data, a gradient boosting classifier outperforms a poorly-featured neural network in almost every operational tabular-data use case. The appropriate starting point for most MSS classification and regression problems is gradient boosting (XGBoost or LightGBM). Reserve neural architectures for large data volumes, complex feature interactions, or unstructured inputs — and coordinate with the AIE prior to deployment.

DRAFT

SECTION 4 — TRAINING DATA IN AN OPERATIONAL CONTEXT

BLUF: A model trained on data that does not represent its deployment environment will fail in that environment. This is especially acute in USAREUR-AF, where peacetime garrison data may not represent high-tempo operational conditions.

4-1. The Representativeness Problem

Training data is a sample from a distribution. The model learns from that distribution. If the deployment environment has a different distribution — different utilization rates, personnel tempo, environmental conditions — the model's predictions will degrade. This is not a bug. It is a mathematical consequence of training on unrepresentative data.

4-2. Key Questions for Training Data Assessment

1. What time period does this data cover? Training on data from 2019–2021 reflects a pre-large-scale-combat-operations (LSCO) readiness posture. That may not reflect current USAREUR-AF sustainment reality.

2. What operational conditions are represented? If training data comes primarily from garrison operations, the model has not seen high-tempo exercise or deployment patterns. Gear utilization rates, maintenance deferral rates, and failure modes differ significantly.

3. Is DEFENDER exercise data a valid proxy for steady-state? DEFENDER exercises generate high-tempo data but represent compressed, surge conditions. A model trained heavily on DEFENDER-period data will see typical months as anomalously low-activity. Stratify training data to reflect the full operational calendar.

4. Are there selection effects in the data? If training data contains only records from units that submitted readiness reports on time, it excludes units with reporting failures — which may disproportionately be units with readiness problems. This selection bias causes the model to underestimate risk in units with poor reporting discipline.

5. Is the labeling accurate? If labels are wrong, the model learns the wrong thing. Maintenance records may be incomplete because deferred maintenance is not always logged at the time of deferral. Validate label accuracy against ground-truth source systems before training.

4-3. Training Data Documentation Requirements

Every production model on MSS must include a Training Data Card:

DRAFT

Field	Content
Data source(s)	Foundry dataset names and versions
Time range	Start and end date of training data
Operational conditions covered	Garrison, field, exercise, deployment
Known exclusions	What is missing and why
Label source and validation	How target labels were generated and verified
Authorization	Data owner confirmation that training use is authorized
Known distribution shift risks	Conditions under which model performance may degrade

VIGNETTE: A team builds a predictive maintenance model using two years of records. 30% of training data comes from four months of DEFENDER exercise activity. The model learns that high-utilization patterns are "normal." Deployed in garrison, it flags nearly every vehicle as low-risk because garrison utilization appears anomalously low. The model is accurate on its training set but operationally useless in production. Fix: stratified sampling weighted to reflect the true operational calendar distribution.

SECTION 5 — MODEL EVALUATION — OPERATIONAL METRICS VS. STATISTICAL METRICS

BLUF: A model achieving 95% accuracy on a held-out test set may still be operationally useless. Statistical metrics describe performance on historical data. Operational metrics describe whether the model actually improves decisions. Translate all evaluation results into operational terms before presenting to any decision-maker.

5-1. The Accuracy Illusion

On an imbalanced dataset — where 95% of vehicles do not require unscheduled maintenance in a given month — a model predicting "no maintenance required" for every vehicle achieves 95% accuracy and has zero operational utility. This is the accuracy paradox in imbalanced classification. **Never report accuracy alone on imbalanced datasets.**

5-2. The Standard Evaluation Framework

For classification problems:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metric	Formula	Operational Meaning
Precision	TP / (TP + FP)	Of vehicles flagged, how many actually need maintenance?
Recall (Sensitivity)	TP / (TP + FN)	Of vehicles that actually needed maintenance, how many did we catch?
F1 Score	2 × (Precision × Recall) / (Precision + Recall)	Balanced score
AUC-ROC	Area under ROC curve	Rank-ordering quality across all decision thresholds

5-3. Asymmetric Error Costs

Use Case	More Costly Error	Rationale
Unscheduled maintenance prediction	False Negative	Missing a failure leads to breakdown during high-tempo operations
Personnel availability forecasting	False Positive	Incorrectly flagging Soldiers as unavailable affects mission planning
Logistics anomaly detection	False Positive	Excessive false alarms cause operators to ignore alerts
Equipment failure before deployment	False Negative	Field failure in deployed environment has severe operational consequences

Once the asymmetric cost is established, tune the classification threshold accordingly. A default 0.5 threshold treats false positives and false negatives as equally costly. In sustainment contexts, lower the threshold to increase recall and reduce false negatives — accepting more false positives as the operational trade-off.

5-4. Minimum Evaluation Bar for Production

- ☐ Evaluated on a held-out test set temporally separated from training data (no future leakage)
- ☐ Performance reported in operational terms, not only statistical metrics
- ☐ Confusion matrix documented for the selected operating threshold
- ☐ Error cost asymmetry assessed and threshold tuned accordingly

DRAFT

- ☐ Performance compared against a baseline (rule-based approach or simple heuristic)
- ☐ Performance stratified by relevant subgroups (theater, unit type, equipment category) to detect subgroup degradation
- ☐ Evaluation results reviewed by a domain SME before submission to C2DAO

5-5. Communicating Model Performance to Commanders

Statistical framing (MLE documentation): "The model achieves 78% recall and 61% precision at the selected threshold of 0.35."

Operational framing (commander briefing): "When the model flags a vehicle as at-risk, that vehicle actually needs maintenance 61% of the time. Of all vehicles that actually needed maintenance, the model correctly identified 78%. Compared to the current manual review process, the model catches approximately 23% more at-risk vehicles per monthly cycle."

The commander decides on the operational framing. The MLE is responsible for producing both.

SECTION 6 — DEPLOYMENT AS A SYSTEM, NOT AN EVENT

BLUF: Deployment is the beginning of the MLOps lifecycle, not the end of the model development lifecycle. A model deployed without monitoring, drift detection, and retraining protocols will silently degrade. Design the operations plan before deployment.

6-1. What Model Drift Is

Model drift is the degradation of model performance over time due to changes in the real-world distribution the model was trained on.

Drift Type	Description	Example
Feature drift (covariate shift)	Distribution of input features changes	New vehicle fleet fielded across USAREUR-AF, introducing maintenance patterns not present in training data
Label drift (concept drift)	Relationship between inputs and target changes	Change in Army maintenance policy alters what triggers an unscheduled maintenance event

6-2. Drift Detection Design

Design drift detection before deployment. At minimum:

Monitoring Target	Detection Method	Alert Threshold
Feature distribution	KL divergence or PSI on each feature vs. training distribution	PSI > 0.2 triggers review
Prediction distribution	Monitor rolling mean/variance of model output	>2 SD shift over 30-day window
Model performance (if labels available)	Rolling precision/recall on validated labels	Drop >5 points from baseline triggers review
Data pipeline freshness	Monitor input dataset last-updated timestamps	Stale data older than defined SLA

NOTE

Adjust PSI thresholds based on operational criticality. Readiness prediction models warrant tighter thresholds (PSI > 0.1 triggers investigation); lower-stakes forecasting may tolerate higher drift (PSI > 0.25). The 0.2 value is a starting heuristic, not a platform standard.

Document alert thresholds and response protocols for each alert before deployment. Alert without a response protocol is noise, not monitoring.

6-3. Retraining Triggers

Acceptable triggers: - Drift alert fired and confirmed by MLE review - Scheduled periodic retraining (quarterly) - Significant operational event that changes the data distribution (new vehicle system fielded, major policy change, post-deployment data now available)

Unacceptable: Ad-hoc retraining without documentation or C2DAO coordination.

Every retraining event is a model version change. Every model version change requires re-validation and C2DAO coordination before entering production.

DRAFT

6-4. The Governance Chain for Model Changes

```
MLE identifies retraining need
         ↓
MLE re-trains and re-validates candidate model
         ↓
MLE documents: what changed, why, evaluation delta vs. prior version
         ↓
Peer review by second MLE or senior data engineer
         ↓
C2DAO coordination: submit updated Model Card and Training Data Card
         ↓
C2DAO approval
         ↓
Staged deployment: canary or shadow mode in non-production first
         ↓
Production promotion with rollback plan documented
```

Minor vs. Major Updates: - **Minor updates** (data refresh with same feature set and architecture, hyperparameter adjustment within documented bounds) may proceed from re-validation through production promotion without repeating full design review, provided the MLE documents that the change is within scope of the existing C2DAO authorization. - **Model changes** (new features added or removed, new algorithm or architecture, retraining from scratch on materially different data) require the full governance sequence including C2DAO coordination. When in doubt, escalate — the C2DAO gate is the authoritative decision point on what constitutes a "model change" vs. a "minor update."

VIGNETTE: A logistics anomaly detection model deployed in Q1 performs well for six months. In Q3, a theater-wide sustainment reorganization changes how requisition records are structured. The input feature distribution shifts significantly. The model's anomaly detection rate drops from 71% to 34% recall. No alert fires because drift monitoring was not configured at deployment. The degradation is discovered four months later during a quarterly review. Root cause: drift detection designed as a future task, not a deployment prerequisite.

SECTION 7 — MODEL-BACKED ONTOLOGY PROPERTIES — THE FOUNDRY-SPECIFIC MENTAL MODEL

BLUF: In MSS/Foundry, a trained model becomes a property on an Ontology Object Type. Every instance of that object — every vehicle, every Soldier, every unit — has a model-generated score attached to it, visible in Workshop dashboards and accessible via OSDK. Design for the case where the model is wrong, because it will be.

DRAFT

7-1. How It Works

A model trained on tabular data is deployed via an inference transform — a scheduled Foundry Transform that loads the model artifact, runs inference, and writes prediction outputs to a downstream dataset. The Ontology Manager then configures a computed Object property that syncs from that prediction dataset. For real-time scoring, the model can be integrated via AIP Logic, which exposes the inference pipeline as a callable function.

Result: every M-ATV in the theater has a `predicted_maintenance_risk_score` property visible in readiness dashboards and queryable through OSDK applications.

7-2. Required Design Decisions Before Deployment

1. Who can see the model output? Model scores are Ontology properties governed by role-based access control. Define access roles before deployment. Not every user who can see vehicle records should see a raw model confidence score — particularly for personnel models.

2. Who can act on the model output? Is this score advisory or decisional? If advisory, what is the human-in-the-loop process? If the model output can trigger an automated Action (e.g., marking a vehicle non-mission-capable), that automation must be approved through the full C2DAO governance process.

3. What happens when the model is wrong? When a Soldier believes the model score is incorrect for their vehicle, there must be a defined override process. How does a commander flag a model prediction as incorrect? How does feedback reach the MLE? How is it incorporated into the next retraining cycle? Without an override process, Soldiers will either trust wrong predictions or stop trusting the system.

4. What is the score's update frequency? Match update frequency to the decision cycle it is designed to support. A daily-updated score is appropriate for a 30-day maintenance forecast. An hourly-updated score may create false precision. A monthly-updated score may be insufficient if vehicle condition changes rapidly.

7-3. The Wrong-Answer Case

Every model produces wrong answers. When a false negative causes a vehicle to fail in the field: - The model will be blamed - Users will lose trust without an explanation and recourse - The MLE must explain why the model scored that vehicle as low-risk (explainability) - The MLE must trace whether the error was due to a feature issue, a drift event, or a labeling error

Build explainability into the model (SHAP values or equivalent feature attribution) and surface that attribution in the Ontology property or Workshop panel. A model that cannot explain its predictions cannot be trusted in operational contexts.

DRAFT

SECTION 8 — BIAS AND FAIRNESS IN OPERATIONAL MODELS

BLUF: When a model's outputs affect personnel assessments, readiness determinations, or resource allocation decisions, bias is an operational and legal concern — not only a technical one.

8-1. Why This Applies to Military ML

Models that affect which Soldiers are assessed as deployment-ready, which units receive priority maintenance resources, or which personnel are flagged for follow-up assessment are making consequential determinations about individual Soldiers. Under Army EO policy, DoD AI ethics principles, and applicable law, those determinations cannot be systematically biased by protected characteristics.

8-2. Direct and Proxy Characteristics

Direct characteristics — explicit features representing protected attributes (gender, race/ethnicity, age, religion, national origin) — must not be included in models generating personnel assessments.

Proxy characteristics are more subtle — features highly correlated with a protected attribute without explicitly measuring it:

Feature	Potential Proxy For
MOS code	Gender (certain MOSs are disproportionately male or female)
Unit assignment	Race/ethnicity (if unit demographics are correlated with assignment patterns)
Deployment history length	Age, family status
Physical fitness test component scores	Gender (scores are normalized by gender; raw scores are not)
Zip code of home record	Race/ethnicity (residential segregation patterns)

Audit feature sets for proxy characteristics before finalizing any model making personnel-adjacent predictions. The presence of a proxy characteristic does not automatically disqualify a feature, but it must be explicitly documented and evaluated for disparate impact.

8-3. Minimum Fairness Evaluation Before Production

1. **Subgroup performance parity check.** Evaluate model precision, recall, and false positive rate stratified by gender, and where data supports it, by rank group and MOS category. A disparity greater than 5 percentage points in false positive rate between subgroups requires review before deployment.

2. **Feature proxy audit.** Document any features flagged as potential proxies for protected characteristics, explain why they are retained or removed, and note the performance impact of removal.
3. **Legal review coordination.** Models affecting personnel determinations require coordination with the Staff Judge Advocate (SJA) assigned to the data team before production deployment. This is a hard requirement.
4. **Adverse impact ratio.** For any binary classification model resulting in different selection rates by group, compute the adverse impact ratio (selection rate of protected group ÷ selection rate of majority group). A ratio below 0.8 (the four-fifths rule) requires documented justification before deployment.

8-4. Fairness Is Not a Post-Hoc Check

Bias mitigation is most effective during feature engineering and training, not after a model is built. If a fairness check at deployment reveals significant disparate impact, the fix often requires retraining. Build fairness evaluation into the development cycle at the feature selection stage.

SECTION 9 — COMMON MLE FAILURE MODES ON MSS

BLUF: The following failure modes recur across ML deployments — caused by time pressure, overconfidence, insufficient domain consultation, and skipping governance steps that feel redundant when the model is performing well.

DRAFT

Failure Mode	Stage	What It Is	Detection	Prevention
Data leakage in feature pipelines	Feature engineering	A feature including future information relative to the prediction point inflates training performance; production performance collapses	Production underperforms evaluation significantly	Temporal train/test split; `.shift(1)` on rolling features
Garrison overfitting	Training data	Model trained and evaluated on garrison data deployed into field exercise or high-tempo period	Field/exercise accuracy collapse	Characterize training data by operational tempo; evaluate on exercise-period data as a separate test partition
No drift detection	Deployment	Model deployed to production with no monitoring; degradation discovered months later with no alert history	Silent degradation discovered during quarterly review	Treat drift monitoring as a deployment prerequisite, not a post-deployment task
Accuracy vs. reliability confusion	Evaluation	Strong overall accuracy metrics mask subgroup performance gaps	Subgroup performance gap discovered post-deployment	Require stratified evaluation before production; report by equipment category, unit type, theater region
Governance bypass under time pressure	Deployment	Urgent requirement generates pressure to skip C2DAO coordination; model deployed directly to production	Policy violation; flawed model in production	Escalate timeline conflicts to leadership; request expedited review; never skip
Training on unauthorized data	Data access	Dataset used for training without confirming data owner authorization for ML training use	Compliance finding	Confirm authorization via email or Jira ticket before including any dataset in a training pipeline

SUMMARY: THE MLE MENTAL MODEL ON MSS

1. **ML is a tool, not a default.** Apply it only where it adds value over simpler methods. The discipline to not build a model is as important as the skill to build one.
2. **Features are hypotheses.** Treat feature engineering as a domain modeling exercise. Consult SMEs. Validate every feature.

3. **Training data defines the model's world.** If the training data does not represent the deployment environment, the model will fail in that environment. Document scope explicitly.
4. **Statistical metrics are not operational metrics.** Translate all evaluation results into commander language before any production decision. Tune thresholds to reflect asymmetric error costs.
5. **Deployment begins the MLOps lifecycle.** Design monitoring and retraining protocols before deployment. Treat drift as a system failure, not a performance metric.
6. **Every model output is operational.** In Foundry, a model score is a property on a real object — a vehicle, a Soldier, a unit. Design for the wrong-answer case. Build explainability in.
7. **Bias is an operational and legal concern.** Audit for proxy characteristics. Run fairness checks. Coordinate with SJA on personnel-adjacent models.
8. **Governance is a safety check, not overhead.** C2DAO coordination catches errors the MLE misses under pressure. Do not bypass it.

This document is a prerequisite companion to TM-40I (ML Engineer). Proceed to TM-40I task instruction upon completion.

DISTRIBUTION RESTRICTION: DRAFT — Not yet approved for distribution.

Document Control: Version 1.0 | 2026 | USAREUR-AF Operational Data Team | Associated Manual: TM-40I

DRAFT